Day 9

Advanced JavaScript

DOM, Events, Array Methods, ES6, Promises, async/await and Fetch

Topics	Tasks	Expected Output
DOM manipulation Event handling map, filter, reduce ES6: arrow functions, destructuring, spread and rest Promises async and await Fetch API basics	Create an interactive webpage Button-click and form-submit events Display dynamic data Array filtering and searching Async operations and API responses	Interactive webpage DOM and array-method practice

Part 1: Topics

1. DOM Manipulation

The DOM (Document Object Model) is a tree representation of the HTML page. JavaScript can select elements, change their content and styles, and add or remove elements.

```
// Selecting elements
const title = document.getElementById("title");
const first = document.querySelector(".item");        // first match
const all   = document.querySelectorAll(".item");     // NodeList

// Changing content and style
title.textContent = "Hello DOM";       // plain text (safe)
title.innerHTML   = "<b>Hello</b>";    // parses HTML (avoid with user input)
title.style.color = "tomato";
title.classList.add("active");
title.classList.toggle("hidden");
title.setAttribute("data-id", "5");

// Creating and adding elements
const li = document.createElement("li");
li.textContent = "New item";
document.querySelector("ul").appendChild(li);
li.remove();                           // remove from page
```

2. Event Handling

Events let the page react to the user: clicks, typing, form submission, key presses and more. Use `addEventListener`.

```
const btn = document.getElementById("btn");

btn.addEventListener("click", () => {
  console.log("Button clicked");
});

// Event object
input.addEventListener("input", (event) => {
  console.log(event.target.value);       // current text
});

// Form submission: stop the page from reloading
form.addEventListener("submit", (event) => {
  event.preventDefault();
  const name = form.elements["name"].value.trim();
});

// Event delegation: one listener for many children
list.addEventListener("click", (e) => {
  if (e.target.matches(".delete")) e.target.parentElement.remove();
});
```

Event	Fires when
click	Element is clicked
submit	Form is submitted
input	Value of input changes (every keystroke)
change	Value committed (select, checkbox)
keydown / keyup	Key pressed / released
DOMContentLoaded	HTML is fully parsed

3. Array Methods: map, filter, reduce

Method	Returns	Purpose
map	New array (same length)	Transform every item
filter	New array (subset)	Keep items that pass a test
reduce	Single value	Combine all items into one result
find	First matching item	Search for one item
some / every	true or false	Check if any / all items match
includes	true or false	Check if value exists

```
const nums = [1, 2, 3, 4, 5, 6];

const doubled = nums.map(n => n * 2);          // [2, 4, 6, 8, 10, 12]
const evens   = nums.filter(n => n % 2 === 0); // [2, 4, 6]
const sum     = nums.reduce((total, n) => total + n, 0);  // 21

const students = [
  { name: "Asha", marks: 82 },
  { name: "Ravi", marks: 45 },
  { name: "Meena", marks: 91 }
];
const names   = students.map(s => s.name);
const passed  = students.filter(s => s.marks >= 50);
const average = students.reduce((t, s) => t + s.marks, 0) / students.length;
const topper  = students.find(s => s.marks > 90);

// Searching (case-insensitive)
const term = "me";
const found = students.filter(s => s.name.toLowerCase().includes(term));

// Chaining
const topNames = students
  .filter(s => s.marks >= 80)
  .map(s => s.name.toUpperCase());
```

4. ES6 Features

let and const replace var: let is re-assignable, const is not. Both are block-scoped. **Template literals** use backticks for strings with embedded expressions.

```
const user = "Asha";
const msg = `Hello ${user}, total = ${2 + 3}`;   // template literal
```

5. Arrow Functions

```
// Traditional
function add(a, b) { return a + b; }

// Arrow function
const addArrow = (a, b) => a + b;        // implicit return
const square   = n => n * n;             // one parameter: no parentheses
const greet    = () => "Hi";             // no parameters
const makeObj  = n => ({ id: n });       // returning an object: wrap in ( )
const total    = (a, b) => {             // block body needs return
  const sum = a + b;
  return sum;
};
```

Arrow functions are shorter and do not have their own `this`; they inherit it from the surrounding scope.

6. Destructuring

```
// Array destructuring
const [first, second, ...others] = [10, 20, 30, 40];

// Object destructuring
const student = { name: "Asha", age: 20, city: "Pune" };
const { name, age } = student;
const { city: town = "Unknown" } = student;   // rename + default value

// In function parameters
function show({ name, age }) { return `${name} (${age})`; }

// Swapping values
let a = 1, b = 2;
[a, b] = [b, a];
```

7. Spread and Rest Operators

Both use three dots. **Spread** expands an array or object into individual items. **Rest** collects remaining items into an array.

```
// Spread
const a = [1, 2], b = [3, 4];
const merged = [...a, ...b];                // [1, 2, 3, 4]
const copy   = [...a];                      // shallow copy
const updated = { ...student, age: 21 };    // copy object, change one field
Math.max(...[5, 9, 2]);                     // 9

// Rest
function sumAll(...numbers) {
  return numbers.reduce((t, n) => t + n, 0);
}
sumAll(1, 2, 3, 4);                         // 10
const { name, ...remaining } = student;     // remaining = { age, city }
```

8. Promises and Asynchronous JavaScript

JavaScript is single-threaded. Slow tasks (network requests, timers) run asynchronously so the page does not freeze. A **Promise** represents a value that will be available later. It has three states: **pending**, **fulfilled** (resolved) and **rejected**.

```
const wait = (ms) => new Promise((resolve, reject) => {
  if (ms < 0) reject(new Error("Invalid time"));
  setTimeout(() => resolve(`Waited ${ms} ms`), ms);
});

wait(1000)
  .then(result => { console.log(result); return wait(500); })
  .then(result => console.log(result))
  .catch(error => console.error(error.message))
  .finally(() => console.log("Done"));

// Run several promises together
Promise.all([wait(300), wait(500)]).then(results => console.log(results));
```

9. async and await

`async/await` is cleaner syntax over promises. An async function always returns a promise; `await` pauses inside it until the promise settles. Use try/catch for errors.

```
async function run() {
  try {
    const r1 = await wait(500);
    console.log(r1);
    const r2 = await wait(300);
    console.log(r2);
```

```
  } catch (error) {
    console.error("Failed:", error.message);
  } finally {
    console.log("Finished");
  }
}
run();
```

10. Fetch API Basics

`fetch(url)` sends an HTTP request and returns a promise for a Response. Call `response.json()` (also a promise) to read the data. Note that fetch only rejects on network failure, so check `response.ok` yourself.

```
async function getUsers() {
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/users");
    if (!response.ok) {
      throw new Error(`HTTP error: ${response.status}`);
    }
    const users = await response.json();
    console.log(users);
    return users;
  } catch (error) {
    console.error("Request failed:", error.message);
  }
}

// POST request
async function createPost() {
  const res = await fetch("https://jsonplaceholder.typicode.com/posts", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ title: "Hello", body: "World", userId: 1 })
  });
  console.log(await res.json());
}
```

Status	Meaning
200	OK
201	Created (successful POST)
404	Not found
500	Server error

Part 2: Tasks (Complete Code)

Project: Interactive Student Directory. One page that loads students from an API, lets the user add students with a form, search and filter them, and shows live statistics. Create three files in one folder: `index.html, style.css, script.js`.

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Student Directory</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h1>Student Directory</h1>
    <button id="themeBtn">Toggle Theme</button>
  </header>

  <main>
    <!-- Task: form-submission event -->
    <section class="panel">
      <h2>Add Student</h2>
      <form id="studentForm">
        <input type="text" id="nameInput" placeholder="Name" required>
        <input type="number" id="marksInput" placeholder="Marks (0-100)"
               min="0" max="100" required>
        <button type="submit">Add</button>
      </form>
      <p id="formMsg"></p>
    </section>

    <!-- Task: searching and filtering -->
    <section class="panel">
      <h2>Search and Filter</h2>
      <div class="controls">
        <input type="text" id="searchInput" placeholder="Search by name...">
        <button class="filter-btn active" data-filter="all">All</button>
        <button class="filter-btn" data-filter="pass">Pass (50+)</button>
        <button class="filter-btn" data-filter="fail">Fail (below 50)</button>
        <button id="sortBtn">Sort by Marks</button>
        <button id="loadBtn">Load from API</button>
      </div>
      <p id="stats"></p>
    </section>

    <!-- Task: dynamic data -->
    <section>
      <p id="status"></p>
      <div id="list" class="cards"></div>
    </section>
  </main>

  <script src="script.js"></script>
</body>
</html>
```

style.css

```
* { margin: 0; padding: 0; box-sizing: border-box; }
body { font-family: "Segoe UI", Arial, sans-serif; background: #f1f5f9; color: #1e293b; }
body.dark { background: #0f172a; color: #e2e8f0; }

header {
```

```
  display: flex; justify-content: space-between; align-items: center;
  background: #1e293b; color: #fff; padding: 16px 24px;
}
main { max-width: 1000px; margin: 20px auto; padding: 0 16px; }

.panel {
  background: #fff; padding: 18px; margin-bottom: 18px;
  border-radius: 10px; box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}
body.dark .panel, body.dark .card { background: #1e293b; }
.panel h2 { margin-bottom: 10px; font-size: 1.1rem; }

form, .controls { display: flex; flex-wrap: wrap; gap: 10px; }
input {
  padding: 9px 12px; border: 1px solid #cbd5e1;
  border-radius: 6px; font-size: 1rem; flex: 1 1 160px;
}
button {
  padding: 9px 14px; border: none; border-radius: 6px;
  background: #2563eb; color: #fff; cursor: pointer; font-size: 0.95rem;
}
button:hover { background: #1d4ed8; }
.filter-btn { background: #64748b; }
.filter-btn.active { background: #d97706; }

.cards { display: flex; flex-wrap: wrap; gap: 16px; }
.card {
  flex: 1 1 220px; background: #fff; padding: 16px;
  border-radius: 10px; box-shadow: 0 2px 8px rgba(0,0,0,0.1);
  border-left: 5px solid #16a34a;
}
.card.fail { border-left-color: #dc2626; }
.card h3 { margin-bottom: 6px; }
.card .delete { background: #dc2626; margin-top: 10px; }

#formMsg { margin-top: 8px; font-weight: 600; }
#stats, #status { margin-top: 10px; font-weight: 600; }
.error { color: #dc2626; }
.success { color: #16a34a; }

@media (max-width: 600px) {
  header { flex-direction: column; gap: 10px; }
}
```

script.js

```
// ---------- State ----------
let students = [
  { id: 1, name: "Asha",  marks: 82 },
  { id: 2, name: "Ravi",  marks: 45 },
  { id: 3, name: "Meena", marks: 91 }
];
let nextId = 4;
let currentFilter = "all";
let searchTerm = "";

// ---------- DOM references ----------
const form        = document.getElementById("studentForm");
const nameInput   = document.getElementById("nameInput");
const marksInput  = document.getElementById("marksInput");
const formMsg     = document.getElementById("formMsg");
const searchInput = document.getElementById("searchInput");
const listEl      = document.getElementById("list");
const statsEl     = document.getElementById("stats");
const statusEl    = document.getElementById("status");
const loadBtn     = document.getElementById("loadBtn");
const sortBtn     = document.getElementById("sortBtn");
const themeBtn    = document.getElementById("themeBtn");

// ---------- Render (dynamic data) ----------
```

```
function getVisibleStudents() {
  return students
    .filter(({ name }) => name.toLowerCase().includes(searchTerm))
    .filter(({ marks }) => {
      if (currentFilter === "pass") return marks >= 50;
      if (currentFilter === "fail") return marks < 50;
      return true;
    });
}

function render() {
  const visible = getVisibleStudents();

  listEl.innerHTML = visible.map(({ id, name, marks }) => `
    <div class="card ${marks < 50 ? "fail" : ""}">
      <h3>${name}</h3>
      <p>Marks: ${marks}</p>
      <p>${marks >= 50 ? "Pass" : "Fail"}</p>
      <button class="delete" data-id="${id}">Delete</button>
    </div>`).join("");

  if (visible.length === 0) {
    listEl.innerHTML = "<p>No students found.</p>";
  }
  updateStats(visible);
}

// ---------- Statistics using reduce, map ----------
function updateStats(list) {
  if (list.length === 0) { statsEl.textContent = "Showing 0 students"; return; }
  const total   = list.reduce((sum, s) => sum + s.marks, 0);
  const average = (total / list.length).toFixed(1);
  const highest = Math.max(...list.map(s => s.marks));
  statsEl.textContent =
    `Showing ${list.length} of ${students.length} | Average: ${average} | Highest: ${highest}`;
}

// ---------- Form submission event ----------
form.addEventListener("submit", (event) => {
  event.preventDefault();
  const name  = nameInput.value.trim();
  const marks = Number(marksInput.value);

  if (name === "" || marks < 0 || marks > 100) {
    formMsg.textContent = "Please enter a valid name and marks (0-100).";
    formMsg.className = "error";
    return;
  }
  students = [...students, { id: nextId++, name, marks }];   // spread
  formMsg.textContent = `${name} added successfully.`;
  formMsg.className = "success";
  form.reset();
  render();
});

// ---------- Search (input event) ----------
searchInput.addEventListener("input", (e) => {
  searchTerm = e.target.value.toLowerCase().trim();
  render();
});

// ---------- Filter buttons (click events) ----------
document.querySelectorAll(".filter-btn").forEach(btn => {
  btn.addEventListener("click", () => {
    document.querySelectorAll(".filter-btn")
            .forEach(b => b.classList.remove("active"));
    btn.classList.add("active");
    currentFilter = btn.dataset.filter;
    render();
  });
```

```
});

// ---------- Sort button ----------
sortBtn.addEventListener("click", () => {
  students = [...students].sort((a, b) => b.marks - a.marks);
  render();
});

// ---------- Delete (event delegation) ----------
listEl.addEventListener("click", (e) => {
  if (e.target.matches(".delete")) {
    const id = Number(e.target.dataset.id);
    students = students.filter(s => s.id !== id);
    render();
  }
});

// ---------- Theme toggle ----------
themeBtn.addEventListener("click", () => document.body.classList.toggle("dark"));

// ---------- Async: fetch from API ----------
async function loadFromApi() {
  statusEl.textContent = "Loading...";
  statusEl.className = "";
  loadBtn.disabled = true;
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/users");
    if (!response.ok) throw new Error(`HTTP error ${response.status}`);
    const users = await response.json();

    // Convert API data into our student format (map + rest of fields ignored)
    const apiStudents = users.map(({ id, name }) => ({
      id: 1000 + id,
      name,
      marks: Math.floor(Math.random() * 51) + 50   // random 50-100
    }));

    students = [...students, ...apiStudents];
    statusEl.textContent = `Loaded ${apiStudents.length} students from API.`;
    statusEl.className = "success";
    render();
  } catch (error) {
    statusEl.textContent = `Could not load data: ${error.message}`;
    statusEl.className = "error";
  } finally {
    loadBtn.disabled = false;
  }
}
loadBtn.addEventListener("click", loadFromApi);

// ---------- First render ----------
render();
```

Part 3: Practice Exercises

A. Array Method Practice (run in the browser console)

```
const products = [
  { name: "Laptop", price: 55000, category: "electronics" },
  { name: "Phone",  price: 20000, category: "electronics" },
  { name: "Shirt",  price: 800,   category: "clothing" },
  { name: "Shoes",  price: 2500,  category: "clothing" }
];

// 1. Get an array of product names                -> map
// 2. Get products costing more than 2000          -> filter
// 3. Calculate total price of all products        -> reduce
// 4. Find the product named "Shirt"               -> find
// 5. Check if any product costs above 50000       -> some
// 6. Total price of only clothing items           -> filter + reduce
// 7. Add 10% discount to every price (new array)  -> map + spread

// Sample solutions
const names   = products.map(p => p.name);
const costly  = products.filter(p => p.price > 2000);
const total   = products.reduce((t, p) => t + p.price, 0);
const shirt   = products.find(p => p.name === "Shirt");
const hasBig  = products.some(p => p.price > 50000);
const clothes = products.filter(p => p.category === "clothing")
                        .reduce((t, p) => t + p.price, 0);
const sale    = products.map(p => ({ ...p, price: p.price * 0.9 }));
```

B. Async Practice

```
// Predict the output order, then run it:
console.log("1. Start");
setTimeout(() => console.log("2. Timeout"), 0);
Promise.resolve().then(() => console.log("3. Promise"));
console.log("4. End");
// Output: 1. Start, 4. End, 3. Promise, 2. Timeout
// (synchronous code first, then promise callbacks, then timers)
```

C. Extra Challenges

- Add an Edit button on each card that updates the student's marks.
- Show a grade (A, B, C, Fail) using if/else or a lookup object.
- Add a dropdown to filter by grade.
- Fetch posts from jsonplaceholder.typicode.com/posts and show the first 10 titles.
- Display a loading message and an error message when the fetch fails (turn off Wi-Fi to test).
- Save the students list in localStorage so it persists after refresh.

Part 4: Expected Output

Interactive Webpage

- Header with a working theme toggle (light/dark).
- Add Student form that validates input and shows a success or error message without reloading the page.
- Live search box that filters cards as you type.
- Filter buttons (All, Pass, Fail) and a Sort by Marks button.
- Delete button on every card using event delegation.

- Load from API button that shows Loading, then the fetched students, or an error message.
- Statistics line showing count, average and highest marks, updated after every change.

DOM and Array-Method Practice

Concept	Where it is used in the project
DOM selection / creation	getElementById, querySelectorAll, innerHTML rendering
Events	click, submit, input, delegation on the list
map	Building card HTML; converting API users to students
filter	Search, pass/fail filter, delete
reduce	Calculating total and average marks
Arrow functions	All callbacks and event handlers
Destructuring	{ name, marks } in callbacks
Spread / rest	Adding items, copying arrays before sort, Math.max(...)
Promises, async/await	loadFromApi with try/catch/finally
Fetch API	GET request to jsonplaceholder users endpoint

Debugging Tips and Revision Notes

- Open DevTools (F12): Console tab for errors, Network tab to inspect fetch requests.
- Put the script tag at the end of body, or use `defer`, so elements exist before JavaScript runs.
- Use `textContent` instead of innerHTML when showing user-typed text, to avoid injecting HTML.
- Always call `event.preventDefault()` in submit handlers.
- map returns a new array; forEach returns nothing.
- Always give reduce an initial value (for example 0).
- fetch does not reject on 404 or 500; check `response.ok`.
- Forgetting await gives you a Promise object instead of the data.
- querySelectorAll returns a NodeList; use forEach or Array.from to loop it.